Custom Schedule Jobs:

In order to have more customization and frequency in the schedule of jobs, Laravel allows the same with the manual CRON expression:

```
$schedule->command('test')->cron('* * * * *');
```

Overlapping Jobs Avoidance:

With a number of jobs being scheduled, it is critical that the previously scheduled completes before the same job is retriggered. To handle this overlapping of jobs, Laravel provides a method that can be used in scheduling:

```
$schedule->command('test')->withoutOverlapping();
```

The **`withoutOverlapping()`** method ensures that the **`test`** command will run every minute if it not running currently. This ensures there is no overlap in the jobs running.

Isolatable Commands:

When working on developing Laravel Applications, as a developer, multiple commands may run. There will be instances when you want only an instance of the command to run at a given time. In this scenario, the concept of Isolatable Commands comes into the picture.

The Isolatable interface consists of all the core logic required to execute Isolatable Commands. The interface needs to be implemented in the command class to use the functionality.

The interface is located at:

```
Illuminate\Contracts\Console\Isolatable
```

This Isolatable interface is used by the class that will implement the interface using the **`implements`** keyword.

Let us understand this concept in depth with the following example.

Consider a scenario where a class named **`SendNotifications`** is to be implemented in such a way that an instance of the class is invoked at any given time in the application. This makes it need the functionality of the **`Isolatable`** interface.